moustaches

Statistiques françaises (tableaux et diagrammes).

v0.1.3 Akilon <https://github.com/Akilon27> MIT

A propos

En faisant mes cours, je me suis rendu compte qu'aucun des paquets pour présenter des diagrammes ne propose la gestion des effectifs ni les définitions françaises des quartiles, ni la construction d'histogrammes, des boîtes-à-moustaches tels que je les ai enseignées … D'où ce paquet.

Je me suis inspiré d'une partie de ProfCollege de LaTeX (auteur : Christophe Poulain) mais l'implémentation n'en est pas une copie et je n'ai pas utilisé du tout l'IA pour ce paquet.

Dans les nouvelles versions, j'ai ajouté la gestion des tableaux de proportionnalité, la résolution d'équation et les opérations sur les matrices (avec des flèches pour les explications et une fonction mblock pour fusionner des cases) et plusieurs fonctions auxiliaires pour ces flèches (à l'aide de fletcher 0.5.8) et une fonction help-fr (à l'aide de tidy 0.4.3).

Partie principale « stat » et module « gridmath »

Toutes les fonctions de cette partie à partir de `math-grid` font partie du module `gridmath` et peuvent être importées séparément.

- simpli()
- pgcd()
- bandes()
- camembert()
- histogramme()
- caracteristiques()
- stat()
- mblock()
- zero-box()
- math-grid()
- lin-up()
- lin-down()
- row-op()
- x-product()
- col-op()
- col-combine()
- rdarrow()
- ruarrow()
- ldarrow()
- luarrow()
- urarrow()
- ularrow()
- drarrow()
- dlarrow()

simpli

Ecrit le quotient a / b avec a et b flottants (ou a si b = 1) et son écriture sous forme de fraction irréductible (max 6 chiffres après la virgule)

```
#simpli(3.6,6.9)
```

$$\frac{3.6}{6.9} = \frac{12}{23}$$

Paramètres

```
simpli(
  a: int float,
  b: int float
) -> content
```

pgcd

Calcule le PGCD d'une liste de nombres

```
#pgcd(1000, 1200, 1400, 1600, 1800,
2000)
```

```
200
```

Paramètres

```
pgcd(..a: array) -> int
```

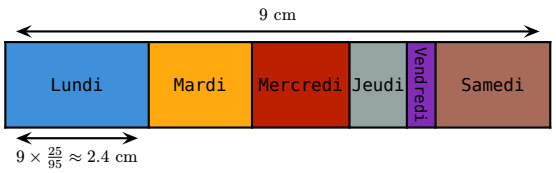
..a `array`

Liste de nombres

bandes

Dessine un diagramme en bandes

```
#bandes(valeurs:("Lundi", "Mardi",
"Mercredi", "Jeudi", "Vendredi",
"Samedi"),effectifs:(25, 18, 17, 10,
5, 20),width:9cm,tourne:
(4,),explication: (0,))
```



Paramètres

```
bandes(
  valeurs: array,
  effectifs: array,
  width: length,
  tourne: array,
  couleurs: auto array,
  print: boolean,
  explication: array,
  hauteur: length
) -> content
```

valeurs `array`

Modalités du caractères

Par défaut: `()`

effectifs `array`

Effectifs de chaque valeur

Par défaut: `()`

width `length`

Longueur totale de la bande

Par défaut: `1fr`

tourne `array`

Pour tourner les labels des rectangles de 90deg

Par défaut: `()`

couleurs `auto` or `array`

Liste des couleurs des rectangles

Par défaut: `auto`

print `boolean`

Si true, remplace les couleurs par des tilings

Par défaut: `false`

explication `array`

Explication de la construction : la ou les étapes à expliquer (0,) ou (0,3)

Par défaut: `()`

hauteur `length`

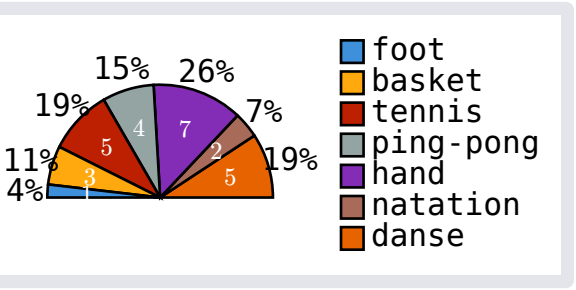
Hauteur de la bande

Par défaut: `40pt`

camembert

Dessine un diagramme (semi-)circulaire

```
#camembert(valeurs:
("foot","basket","tennis","ping-
pong","hand","natation","danse"),effecti
(1, 3, 5, 4, 7, 2,
5),radius:1.5,legende: "d",espace:
0,semi: true)
```



Paramètres

```
camembert(
  valeurs: array,
  effectifs: array,
  data: array,
  semi: boolean,
  legende: str,
  espace: int float,
  print: boolean,
  couleurs: auto array,
  dedans: boolean none dictionnary,
  dehors: boolean none dictionnary,
  ..args: arguments
) -> content
```

valeurs `array`

Modalités du caractères

Par défaut: `()`

effectifs `array`

Effectifs de chaque valeur

Par défaut: `()`

data `array`

Pour garder la possibilité de ne saisir qu'une liste de valeurs

Par défaut: `()`

semi `boolean`

Si true, dessine un diagramme semi-circulaire

Par défaut: `false`

legende `str`

Position de la légende : « d », « g », « h », « b » sinon « « rien

Par défaut: `"d"`

espace `int` or `float`

Espacement des items dans la légende si à gauche ou droite

Par défaut: `.1`

print `boolean`

Si true, remplace les couleurs par des tilings

Par défaut: `false`

couleurs `auto` or `array`

Liste des couleurs des secteurs

Par défaut: `auto`

dedans `boolean` or `none` or `dictionnary`

True, none, false ou passe à inner-label

Par défaut: `true`

dehors `boolean` or `none` or `dictionnary`

True, none, false ou passe à outer-label

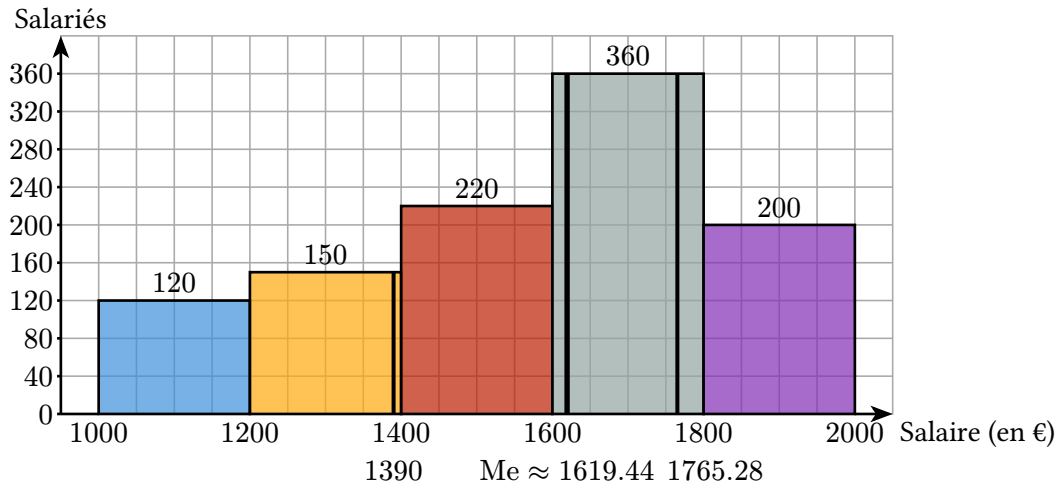
Par défaut: `true`

..args `arguments`

Autres arguments passés à la fonction piechart de cetz

histogramme
Dessine un VRAI histogramme : les classes n'ont pas forcément la même amplitude et ce sont les aires qui sont proportionnelles

```
#histogramme(
   classes: (1000, 1200, 1400, 1600, 1800, 2000),
   effectifs: (120, 150, 220, 360, 200),
   px: 50,
   uaire: 10,
   s: .5,
   // textsize: .5em,
   nom-donnee: [Salaire (en €)],
   nom-effectifs: [Salariés],
   mediane: true,
   quartiles: true,
   pos-rect: (0,5),
 )
```



Paramètres

```
histogramme(
  classes: array,
  effectifs: array,
  px: int,
  uaire: int,
  s: int float dictionnary,
  grille: boolean,
  pas-grille: int float,
  nom-donnee: content,
  nom-effectifs: content,
  add: int array,
  lecture: boolean,
  donnees-sup: boolean,
  liste-couleurs: auto color array,
  pos-rect: array,
  print: boolean,
  transparence: ratio,
  ecc: boolean,
  mediane: boolean,
  quartiles: boolean,
  decimales: int,
  posq1: array,
  posmediane: array,
  posq3: array,
  stroke-mediane: length stroke,
  stroke-quartiles: length stroke,
  textsize: length
) -> content
```

classes `array`

Liste des bornes des classes : longueur = longueur de `effectifs` + 1

Par défaut: `()`

effectifs `array`

Effectifs de chaque classe : longueur = longueur de `classes` - 1

Par défaut: `()`

px `int`

Pas sur l'axe des abscisses

Par défaut: 1

uaire `int`

Quantité représentée par un carreau

Par défaut: `1`

s `int` or `float` or `dictionnary`

Echelle cetz du graphique

Par défaut: `1`

grille `boolean`

Affichage ou non d'un quadrillage

Par défaut: `true`

pas-grille `int` or `float`

Pas du quadrillage

Par défaut: `1`

nom-donnee `content`

Nom pour la légende ou l'axe des ordonnées le cas échéant

Par défaut: `[Valeurs]`

nom-effectifs `content`

Nom pour l'axe des abscisses

Par défaut: `[effectifs]`

add `int` or `array`

Combien « ajouter » à gauche, droite et au dessous, soit un triplet, soit un entier unique

Par défaut: `(-1, 1, 1)`

lecture `boolean`

Pour avoir ou non un axe des ordonnées lorsque les classes ont la même amplitude

Par défaut: `true`

donnees-sup `boolean`

Affichage ou non des effectifs au dessus des rectangles

Par défaut: `true`

liste-couleurs `auto` or `color` or `array`

Liste des couleurs des rectangles ou une unique couleur white pour l'impression par ex

Par défaut: `auto`

pos-rect `array`

Position du rectangle de légende par rapport au coin hg du plus petit rectangle

Par défaut: `(0, 2)`

print `boolean`

Si true, remplace les couleurs par des tilings

Par défaut: `false`

transparence `ratio`

Transparence des rectangles : mettre à 100% si liste-couleurs: white

Par défaut: `30%`

ecc `boolean`

Affichage des effectifs cumulés croissants

Par défaut: `false`

mediane `boolean`

Affichage ou non de la médiane

Par défaut: `false`

quartiles `boolean`

Afficha ou non des quartiles

Par défaut: `false`

decimales `int`

Nombre de décimales des arrondis

Par défaut: `2`

posq1 `array`

Position de la valeur de q1 par rapport à l'axe des abscisses (0,-1.5) par défaut

Par défaut: `(0,-1.5)`

posmediane `array`

Position de la valeur de la médiane par rapport à l'axe des abscisses (0,-1.5) par défaut

Par défaut: `(0,-1.5)`

posq3 `array`

Position de la valeur de q3 par rapport à l'axe des abscisses (1,-1.5) par défaut

Par défaut: `(1,-1.5)`

stroke-mediane `length` or `stroke`

Trait pour représenter la médiane 2pt par défaut

Par défaut: `2pt`

stroke-quartiles `length` or `stroke`

Trait pour représenter les quartiles 2pt par défaut

Par défaut: `1.5pt`

textsize `length`

Taille des textes

Par défaut: `1em`

caracteristiques
Produit un dictionnaire des caractéristiques : effectif-total, mediane, classe-mediane, q1, q3, d1, d9, ecart-interquartiles, max, min, etendue, modes, effectif-modes, sommex, sommex2, moyenne, variance, ecart-type, variance-echantillon, ecart-type-echantillon, lqboxplot, cetzboxplot

```
#caracteristiques(sondage: (1, 5, 7,
9, 4, 5, 9, 8, 7, 3, 8),)
```

```
(
  effectif-total: 11,
  mediane: 7,
  classe-médiane: none,
  q1: 4,
  q3: 8,
  d1: 3,
  d9: 9,
  ecart-interquartiles: 4,
  max: 9,
  min: 1,
  etendue: 8,
  modes: (5, 7, 8, 9),
  effectif-modes: 2,
  sommex: 66,
  sommex2: 464,
  moyenne: 6.0,
  variance: 6.18181818181818,
  ecart-type: 2.4863262420322436,
  variance-echantillon: 6.799999999999997,
  ecart-type-echantillon: 2.607680962081059,
  lqboxplot: (
    median: 7,
    q1: 4,
    q3: 8,
    whisker-low: 3,
    whisker-high: 9,
    outliers: (1,),
    mean: 6.0,
  ),
  cetzboxplot: (
    q1: 4,
    q2: 7,
    q3: 8,
    min: 3,
    max: 9,
    outliers: (1, 6.0),
  ),
)
```

Paramètres

```
caracteristiques(
  valeurs: array,
  effectifs: array,
  classes: array,
  bins: int,
  sondage: array,
  crochets: boolean,
  deciles: boolean
) -> array
```

valeurs `array`

Modalités du caractère numérique

Par défaut: `()`

effectifs `array`

Effectifs de chaque valeur

Par défaut: `()`

classes `array`

Bornes des classes le cas échéant : longueur = longueur effectifs + 1

Par défaut: `()`

bins `int`

Pour un sondage, création automatique des classes de même amplitude si bins > 0

Par défaut: `0`

sondage `array`

Liste de toutes les valeurs dans le cas d'un sondage

Par défaut: `()`

crochets `boolean`

Pour les des classes et donc de la classe médiane, soit avec des crochets soit des inégalités a <= ... < b

Par défaut: `false`

deciles `boolean`

Prise en compte ou non des déciles pour les moustaches renvoyées dans lqboxplot et cetzboxplot, sinon ±1.5 IQR

Par défaut: `true`

stat

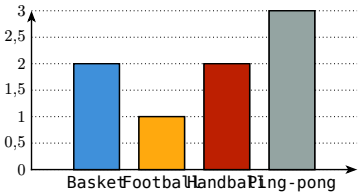
Fait l'étude statistique et présente les résultats, soit sous forme de tableau avec possibilité d'afficher les `effectifs`, `fréquences` (« f » pour fractions, « d » pour décimales, « t » pour les trois, true ou autre pour %), `angles` pour les diagrammes (semi)-circulaires, `ecc`, `fcc`, `classes` (automatiques si `bins` > 0) et leurs `centres`, et les `diagrammes` : si « hbar » ou « bar » ou « circ » ou « semicirc » ou « box » ou « bande » ou « histo » ou une combinaison dans `diagramme` avec des paramétrages : `liste-couleurs` (petroff10 par défaut) ... Si `multi` != (), possibilité d'afficher un diagramme en barres (h ou v) ou des boites à moustaches avec une option `print` pour avoir des tilings

```
#stat(
  classes: (150, 160, 170, 200),
  effectifs: (3, 4, 5),
  frequences: "f",
)
```

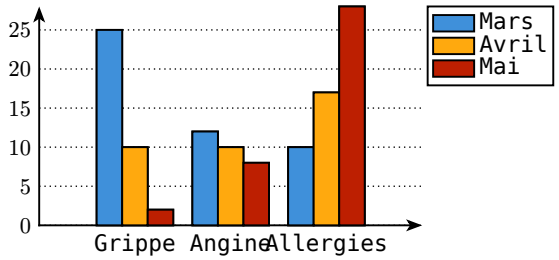
	$150 \leqslant ... < 160$	$160 \leqslant ... < 170$	$170 \leqslant ... < 200$
Centres des classes	155	165	185
Effectifs	3	4	5
Fréquences	$\frac{3}{12} = \frac{1}{4}$	$\frac{4}{12} = \frac{1}{3}$	$\frac{5}{12}$

```
#stat(sondage:
("Handball","Basket","Football","Handbal
pong","Basket","Ping-pong","Ping-
pong"),diagramme: "bar",cbar:(size:
(8.2,4)))
```

	Basket	Football	Handball	Ping-pong
Effectifs	2	1	2	3
Fréquences (en %)	25	13	25	38




```
#stat(valeurs:
("Grippe","Angine","Allergies"),multi:
((25,12,10),(10,10,17),
(2,8,28),),diagramme: "bar",labels:
("Mars","Avril","Mai",),tableau:
false)
```



Paramètres

```
stat(
  valeurs: array,
  effectifs: array,
  qualitatif: boolean,
  totaux: boolean,
  tableau: boolean,
  couleur-tableau: color,
  inset: length,
  nom-donnee: content,
  nom-effectifs: content,
  decimales: int,
  frequences: boolean str,
  sondage: array,
  vide: boolean,
  angle: boolean str,
  ecc: boolean str,
  classes: array,
  bins: int,
  crochets: boolean,
  centre: boolean str,
  colonnes-vide: array,
  cases-vide: array,
  diagramme: str,
  bar: array,
  cbar: array,
  moustaches: array,
  cmoustaches: array,
  circ: dictionnary,
  bande: array,
  histo: array,
  fcc: boolean str,
  multi: array,
  labels: array,
  print: boolean,
  liste-couleurs: auto color array
) -> content
```

valeurs `array`

Modalités du caractère `numérique`

Par défaut: `()`

effectifs `array`

Effectifs de chaque valeur

Par défaut: `()`

qualitatif `boolean`

Type de caractère

Par défaut: `true`

totaux `boolean`

Affichage des totaux dans les tableaux

Par défaut: `false`

tableau `boolean`

Affichage d'un tableau

Par défaut: `true`

couleur-tableau `color`

Couleur de la première ligne et colonne du tableau

Par défaut: `luma(85%)`

inset `length`

Inset du tableau

Par défaut: `5pt`

nom-donnee `content`

Nom des données pour la case hg du tableau (et certains graphiques)

Par défaut: `[]`

nom-effectifs `content`

Nom des effectifs pour la ligne concernée (et certains graphiques)

Par défaut: `[Effectifs]`

decimales `int`

Nombre de décimales des arrondis

Par défaut: `0`

frequences `boolean` or `str`

false/true pour %, « f » pour fraction, « d » pour décimaux, « t » pour tout ou « v » pour une ligne vide

Par défaut: `true`

sondage `array`

Toutes les valeurs une à une pour un sondage

Par défaut: `()`

vide `boolean`

Si true, toutes les lignes sauf la première sont vides

Par défaut: `false`

angle `boolean` or `str`

false/true ou « v » pour une ligne vide

Par défaut: `false`

ecc `boolean` or `str`

false/true ou « v » pour une ligne vide

Par défaut: `false`

classes `array`

Bornes des classes dans le cas d'un caractère continu, longueur = longueur (+1)

Par défaut: `()`

bins `int`

Dans le cas d'un sondage, si bins > 0, alors bins classes sont formées automatiquement

Par défaut: `0`

crochets `boolean`

Dans le cas des classes, affichage par défaut a <= ... < b, si crochets:true, [a ; b[

Par défaut: `false`

centre `boolean` or `str`

false/true ou « v » pour une ligne vide

Par défaut: `true`

colonnes-vide `array`

Pour vider les colonnes correspondantes de toutes leurs valeurs

Par défaut: `()`

cases-vide `array`

Pour vider une liste de cases, numérotation dans la partie blanche, ligne par ligne

Par défaut: `()`

diagramme `str`

« hbar » ou « bar » ou « circ » ou « semicirc » ou « box » ou « bande » ou « histo » ou une combinaison

Par défaut: `""`

bar `array`

paramètres à passer au canvas de bar ou hbar

Par défaut: `()`

cbar `array`

paramètres à passer à barchart ou columnchart de cetz

Par défaut: `()`

moustaches `array`

paramètres à passer au canvas des boites à moustaches

Par défaut: `()`

cmoustaches `array`

paramètres à passer à boxwhiskers de cetz

Par défaut: `()`

circ `dictionnary`

paramètres à passer à la fonction camembert

Par défaut: `(:)`

bande `array`

paramètres à passer à la fonction bandes

Par défaut: `()`

histo `array`

paramètres à passer à la fonction histogramme

Par défaut: `()`

fcc `boolean` or `str`

false/true ou « v » pour une ligne vide

Par défaut: `false`

multi `array`

si multi != (), possibilité d'afficher un diagramme en barres (h ou v) ou des boites à moustaches

Par défaut: `()`

labels `array`

Labels des différentes séries dans le cas de `multi`

Par défaut: `()`

print `boolean`

false/true pour l'affichage de tilings à la place des couleurs dans les graphiques

Par défaut: `false`

liste-couleurs `auto` or `color` or `array`

Liste des couleurs pour les graphiques

Par défaut: `auto`

mblock
Définit une cellule fusionnée ou personnalisée au sein de `math-grid`.

Reçoit les arguments positionnels et nommés :
- **Positionnels** :
  - `body` (content) : Le contenu à afficher dans la cellule.
  - `colspan` (int, optionnel) : Nombre de colonnes occupées (si un seul entier est fourni).
  - `rowspan` (int, optionnel) : Nombre de lignes occupées (si un second entier est fourni).
  - `align` (alignment, optionnel) : Alignement spécifique du contenu dans le bloc.
- **Nommés réservés** :
  - `colspan` (int, défaut: 1) : Extension horizontale de la cellule.
  - `rowspan` (int, défaut: 1) : Extension verticale de la cellule.
  - `align` (alignment, défaut: auto) : Alignement interne du contenu (`left`, `center`, `right`, etc.).
- **Nommés supplémentaires** (`..node-args`) :
  - Tout autre argument nommé (ex: `stroke`, `fill`, `corner-radius`, `extrude`, `def-draw`, etc.) sera transmis directement au nœud Fletcher (`node(...)`) sous-jacent.

-> arguments

Paramètres

```
mblock(..args: arguments) -> content + metadata
```

..args `arguments`

arguments positionnels (body, colspan, rowspan, align, size) et nommés (colspan, rowspan, align, size, ..node-args)

zero-box
Fonction auxiliare pour placer un contenu dans une boite sans prendre de place (avec alignement positionnel ou nommé)

Paramètres

```
zero-box(..args: arguments) -> content
```

..args `arguments`

Contenu et alignement

math-grid

Toutes les fonctions suivantes font partie du module « gridmath »

Dessine un **tableau** de proportionnalité en générant des nœuds Fletcher nommés à partir d'une équation (contenant des alignements ou une matrice), par défaut, tous les nœuds d'une même colonne ont la même largeur et ceux d'une même ligne ont la même hauteur, de plus, tous les nœuds sont collés, fonctionne comme diagram (ou mat) en mode mathématique.

De plus, cette fonction peut être utilisée pour les résolutions d'équations (avec des flèches pour les explications) avec `equation:true` (voir l'exemple du README) ou pour les matrices avec `matrix-mode:true` ou `matrix-mode:"[]"` etc.

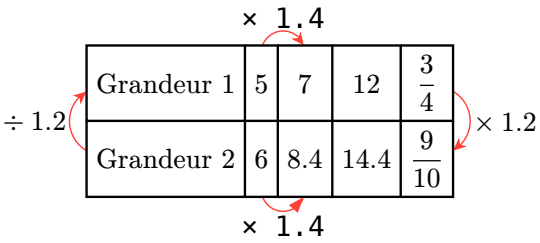
```
// Diagramme fletcher avec les mêmes paramètres que les valeurs par défaut de la fonction proportionnalite
#diagram($
  "Grandeur 1" & 5 & 7   & 9    & 3/4 \
  "Grandeur 2" & 6 & 8.4 & 10.8 & 9/10
$,
  node-stroke: 1pt,
  node-shape: rect,
  spacing: 0cm,
  mark-scale: 1.2,
  node-inset: 6pt,)
```

Grandeur 1	5	7	9	$\frac{3}{4}$
Grandeur 2	6	8.4	10.8	$\frac{9}{10}$

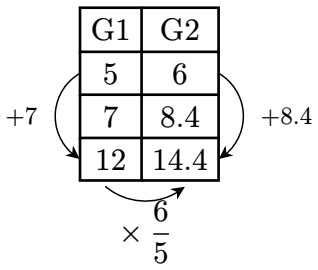
```
#math-grid($
  "Grandeur 1" & 5 & 7   & 9    & 3/4 \
  "Grandeur 2" & 6 & 8.4 & 10.8 & 9/10
$,
)
```

Grandeur 1	5	7	9	$\frac{3}{4}$
Grandeur 2	6	8.4	10.8	$\frac{9}{10}$

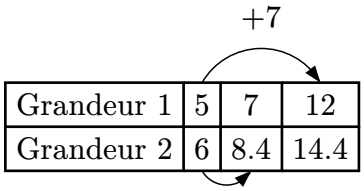
```
#math-grid(
  $ "Grandeur 1"  & 5 edge("r", "-}>",
  bend: #70deg, label: times 1.4, label-sep: #(-1pt),shift:#(5pt)) & 7 & 12 & 3/4  \
  "Grandeur 2" & 6 edge("r", "-|>",
  bend: #(-70deg), label: times 1.4,
  label-sep: #(-1pt),shift:#(-5pt)) &
  8.4 & 14.4 & 9/10 $,
edge-stroke: .6pt+red, coef: $times 1.2$, coef-recip: $div 1.2$,
node-stroke: 1pt)
```



```
#math-grid($"G1" & 5 & 7 & 12 \ "G2" &
6 & 8.4 & 14.4$,
  // change l'orientation à un tableau
  vertical
  flip:true,
  // Arc (au-dessus) à gauche liant la
  (colonne) ligne 1 et la (colonne)
  ligne 3
  col-op(0, 1, 3, $+ 7$, flip:true),
  // Arc (en dessous) à droite liant la
  (colonne) ligne 1 et la (colonne)
  ligne 3
  col-op(1, 1, 3, $+ 8.4$, flip:true),
  coef: $times 6/5$,coef-args: (label-
  sep:2.5pt))
```




```
// Les cases sont automatiquement
nommées <colonne-ligne>
#math-grid(
  $
    "Grandeur 1" & 5 & 7   & 12 \
    "Grandeur 2" & 6 & 8.4 & 14.4
  $,
  // Relie le haut de la cellule (1,0)
  au haut de la cellule (3,0)
  edge(<1-0.north>, <3-0.north>, "-|>",
  bend: 60deg, label: $+ 7$),
  // Relie le bas-droite de (1,1) au
  bas-gauche de (2,1)
  edge(<1-1.south>, <2-1.south>, "-|>",
  bend: -60deg),
)
```



Paramètres

```
math-grid(
  eq: math.equation,
  equation: boolean,
  matrix-mode: boolean array str,
  matrix-sep: length,
  augment: none int array dictionary,
  display: boolean,
  equal-columns: boolean,
  equal-rows: boolean,
  node-stroke: none length color stroke,
  cadre: boolean,
  node-align: auto alignment,
  node-shape: function,
  inset: array,
  spacing: length,
  mark-scale: int float,
  flip: boolean,
  coef: none content,
  coef-recip: none content,
  coef-args: dictionary,
  coef-recip-args: dictionary,
  no-empty-node: boolean,
  fill-empty: boolean color,
  min-width: length,
  min-height: length,
  fill-map: dictionary function,
  header-row: boolean,
  header-col: boolean,
  header-fill: color,
  header-align: none alignment,
  clean-grid: boolean,
  ..args: arguments
) -> content
```

eq `math.equation`

équation mathématique qui sera traduite en tableau, soit comme dans fletcher soit dans une matrice (pour avoir une matrice de largeur usuelle, il faut donner column-gap:`0.2em`)

equation `boolean`

mode équation pour avoir l'alternance des alignements, pas de traits et pas de nœuds vides

Par défaut: `false`

matrix-mode `boolean` or `array` or `str`

mode matrix pour avoir les parenthèses ou crochets, pas de traits etc si true, des parenthèses par défaut, sinon le ou les (noms des) symbole(s) peuvent être donnés comme dans la fonction mat de typst.

Par défaut: `false`

matrix-sep `length`

espacement des parenthèses ou crochets etc

Par défaut: `0pt`

augment `none` or `int` or `array` or `dictionary`

Dessine des lignes d'augmentation dans une matrice, comme dans la fonction mat de typst.

Par défaut: `none`

display `boolean`

force le mode d'affichage mathématique grand format (display)

Par défaut: `true`

equal-columns `boolean`

booléen pour indiquer que toutes les colonnes doivent être de même largeur

Par défaut: `false`

equal-rows `boolean`

booléen pour indiquer que toutes les lignes doivent être de même hauteur

Par défaut: `false`

node-stroke `none` or `length` or `color` or `stroke`

caractéristiques des traits du tableau

Par défaut: `1pt`

cadre `boolean`

booléen pour indiquer s'il faut tracer le cadre extérieur du tableau

Par défaut: `true`

node-align `auto` or `alignment`

alignement à l'intérieur des noeuds

Par défaut: `auto`

node-shape `function`

forme des nœuds

Par défaut: `rect`

inset `array`

inset (h,v) des nœuds en nombre de points ou en donnant explicitement des longueurs

Par défaut: `(4, 4)`

spacing `length`

espacement des cases du tableau

Par défaut: `0cm`

mark-scale `int` or `float`

échelle des marques des flèches

Par défaut: `1.2`

flip `boolean`

booléen pour mettre le tableau vertical

Par défaut: `false`

coef `none` or `content`

coefficient de proportionnalité (de la 1ère ligne vers la 2nde)

Par défaut: `none`

coef-recip `none` or `content`

coefficient réciproque (de la 2ème ligne vers la 1ère)

Par défaut: `none`

coef-args `dictionary`

arguments facultatifs à passer à la fonction `edge` pour le coefficient

Par défaut: `()`

coef-recip-args `dictionary`

arguments facultatifs à passer à la fonction `edge` pour le coefficient réciproque

Par défaut: `()`

no-empty-node `boolean`

booléen pour indiquer de dessiner les nœuds vides

Par défaut: `true`

fill-empty `boolean` or `color`

Pour donner une couleur aux nœuds vides

Par défaut: `false`

min-width `length`

largeur minimale (utile pour les colonnes vides)

Par défaut: `0pt`

min-height `length`

hauteur minimale (utile pour les lignes vides)

Par défaut: `0pt`

fill-map `dictionary` or `function`

dictionnaire « x-y »: color ou fonction (x, y) => color pour colorer des cellules

Par défaut: `(:)`

header-row `boolean`

applique la couleur d'en-tête sur la première ligne (y = 0)

Par défaut: `false`

header-col `boolean`

applique la couleur d'en-tête sur la première colonne (x = 0)

Par défaut: `false`

header-fill `color`

couleur appliquée aux en-têtes

Par défaut: `rgb("d0d0d0")`

header-align `none` or `alignment`

pour appliquer un alignement différent aux titres (x = 0 ou y = 0)

Par défaut: `none`

clean-grid `boolean`

évite le doublement de l'épaisseur des bordures internes quand spacing = 0cm

Par défaut: `true`

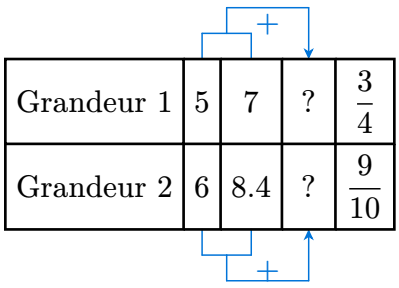
..args `arguments`

arguments facultatifs à passer à la fonction `diagram` de Fletcher

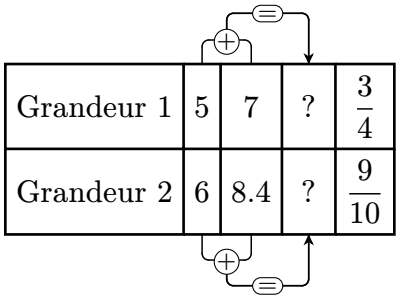
lin-up

Fonction pour montrer qu'une colonne est la somme (défaut) ou la différence de 2 autres, au-dessus de la première ligne, 3 modes sont proposés

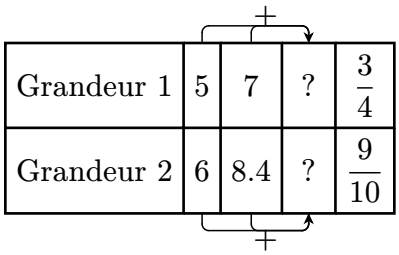
```
#math-grid(
  $ "Grandeur 1"  & 5  & 7 & ? & 3/4  \
  "Grandeur 2" & 6 & 8.4 & thick ? thick
  & 9/10 $,
lin-up(1,2,3,sign: $ + $,color: blue,),
lin-down(1,2,3,sign: $ + $, color:
blue,),)
```



```
#math-grid(
  $ "Grandeur 1"  & 5  & 7 & ? & 3/4  \
  "Grandeur 2" & 6 & 8.4 & thick ? thick
  & 9/10 $,
lin-up(1,2,3,sign: $ + $,mode:2,),
lin-down(1,2,3,sign: $ + $, mode:2,),)
```




```
#math-grid(
  $ "Grandeur 1"  & 5  & 7 & ? & 3/4  \
  "Grandeur 2" & 6 & 8.4 & thick ? thick
  & 9/10 $,
lin-up(1,2,3,sign: $ + $,mode:3,),
lin-down(1,2,3,sign: $ + $, mode:3,),)
```



Paramètres

```
lin-up(
  a: int,
  b: int,
  c: int,
  sign: content str,
  color: auto color,
  text-color: auto color,
  mode: int,
  height: length,
  ..args: dictionary
) -> content
```

a `int`

numéro de la première colonne de départ

b `int`

numéro de la seconde colonne de départ

c `int`

numéro de la colonne d'arrivée

sign `content` or `str`

signe de l'opération

Par défaut: `$ + $`

color `auto` or `color`

couleur des flèches

Par défaut: `auto`

text-color `auto` or `color`

couleur du texte

Par défaut: `auto`

mode `int`

3 modes sont proposés

Par défaut: `1`

height `length`

« hauteur » des traits du 1er mode

Par défaut: `7.24pt`

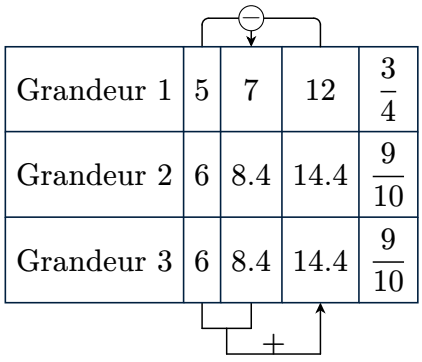
..args `dictionary`

arguments supplémentaires pour les edges

lin-down

Fonction pour montrer qu'une colonne est la somme (défaut) ou la différence de 2 autres, en-dessous de la seconde ligne (ou `n:5` si 5 lignes), 3 modes sont proposés

```
#math-grid(
$"Grandeur 1" & 5 & 7   & 12   & 3/4 \
 "Grandeur 2" & 6 & 8.4 & 14.4 & 9/10 \
 "Grandeur 3" & 6 & 8.4 & 14.4 & 9/10$,
   lin-up(1,3,2,sign: $ - $,mode:2),
   lin-down(1,2,3,n:3),
   node-stroke: 0.6pt+navy,
)
```



Paramètres

```
lin-down(
  a: int,
  b: int,
  c: int,
  sign: content str,
  color: auto color,
  text-color: auto color,
  mode: int,
  n: int,
  height: length,
  ..args: dictionary
) -> content
```

a `int`

numéro de la première colonne de départ

b `int`

numéro de la seconde colonne de départ

c `int`

numéro de la colonne d'arrivée

sign `content` or `str`

signe de l'opération

Par défaut: `$ + $`

color `auto` or `color`

couleur des flèches

Par défaut: `auto`

text-color `auto` or `color`

couleur du texte

Par défaut: `auto`

mode `int`

3 modes sont proposés

Par défaut: `1`

n `int`

nombre de lignes

Par défaut: `2`

height `length`

« hauteur » des traits du 1er mode

Par défaut: `7.24pt`

..args `dictionary`

arguments supplémentaires pour les edges

row-op
Fonction pour faire des opérations sur les lignes (d'une matrice ou autre) avec des flèches à angles droits

Paramètres

```
row-op(
  a: int,
  b: none int,
  c: int,
  label: content str,
  color: auto color,
  text-color: auto color,
  n: int,
  width: length,
  ..args: dictionary
) -> content
```

a `int`

numéro de la première ligne de départ

b `none` or `int`

numéro de la seconde ligne de départ

Par défaut: `0`

c `int`

numéro de la ligne de d'arrivée

label `content` or `str`

label de l'opération

color `auto` or `color`

couleur des flèches

Par défaut: `auto`

text-color `auto` or `color`

couleur du texte

Par défaut: `auto`

n `int`

nombre de colonnes

Par défaut: `2`

width `length`

« largeur » des traits

Par défaut: `3em`

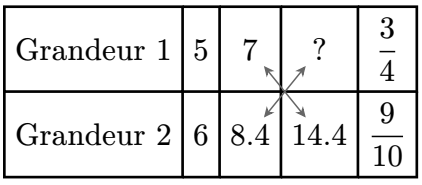
..args `dictionary`

arguments supplémentaires pour les edges

x-product

Fonction pour afficher les croix d'un produit en croix entre deux colonnes

```
#math-grid(
  $ "Grandeur 1"  & 5 & 7 & ? & 3/4  \
  "Grandeur 2" & 6 & 8.4 & 14.4 & 9/10
  $,
x-product(2,),)
```



Paramètres

```
x-product(
  color: color,
  sep: float,
  left-shift: float,
  right-shift: float,
  flip: boolean,
  ..a: int arguments
) -> content
```

color `color`

couleur des flèches

Par défaut: `luma(40%)`

sep `float`

écart avec les centres des nœuds

Par défaut: `.2`

left-shift `float`

décalage des extrémités gauches des flèches si besoin

Par défaut: `0`

right-shift `float`

décalage des extrémités droites des flèches si besoin

Par défaut: `0`

flip `boolean`

pour l'utilisation dans les tableaux verticaux

Par défaut: `false`

..a `int` or `arguments`

arguments contenant nécessairement le numéro de la première colonne et accessoirement celui d'une seconde colonne (sinon +1), ainsi que des arguments transmis à la fonction edge de Fletcher

col-op

Trace une flèche d'opération directe d'une colonne vers une autre au-dessus ou en dessous du tableau.

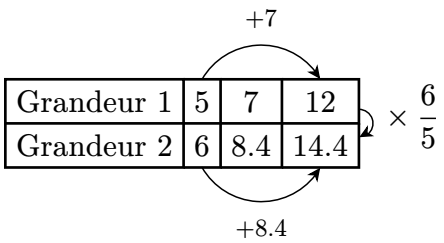
- `row` : indice de la ligne (0 ou 1)
- `f` : indice de la colonne de départ (ex: 1)
- `t` : indice de la colonne d'arrivée (ex: 3)
- `label` : libellé de l'opération (ex: $+7$ ou $+$ Col 2)

et accessoirement :

- `pos` : « top » (au-dessus) ou « bottom » (en dessous)
- `bend` : courbure de l'arc
- `flip` : pour indiquer que le tableau est vertical

1. d'autres arguments pour la commande edge de Fletcher

```
#math-grid($"Grandeur 1" & 5 & 7 & 12 \
"Grandeur 2" & 6 & 8.4 & 14.4$,
  // Arc (au-dessus) à gauche liant la
  (colonne) ligne 1 et la (colonne)
  ligne 3
  col-op(0, 1, 3, $+ 7$),
  // Arc (en dessous) à droite liant la
  (colonne) ligne 1 et la (colonne)
  ligne 3
  col-op(1, 1, 3, $+ 8.4$),
  coef: $times 6/5$,coef-args: (label-
  sep:2.5pt,bend:90deg))
```



Paramètres

```
col-op(
  row: int,
  f: int,
  t: int,
  label: content str,
  pos: auto str,
  bend: auto angle,
  flip: boolean,
  ..args: arguments dictionary
) -> content
```

row `int`

indice de la ligne (ou colonne si flip:true) de départ

f `int`

indice de la colonne (ou ligne si flip:true) de départ

t `int`

indice de la colonne (ou ligne si flip:true) d'arrivée

label `content` or `str`

texte/opération (ex: +12 ou + col 2)

pos `auto` or `str`

position : « top » (au-dessus) ou « bottom » (en dessous), défaut : auto

Par défaut: `auto`

bend `auto` or `angle`

courbure de l'arc

Par défaut: `auto`

flip `boolean`

pour l'utilisation dans les tableaux verticaux

Par défaut: `false`

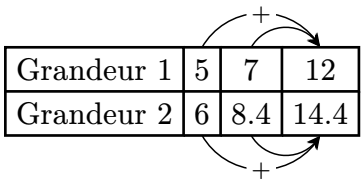
..args `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

col-combine

Double flèche montrant la combinaison (addition/soustraction) de deux colonnes vers une 3e.

```
#math-grid(
  $
    "Grandeur 1" & 5 & 7 & 12 \
    "Grandeur 2" & 6 & 8.4 & 14.4
  $,
  col-combine(0, 1, 2, 3, label: $+$,),
  col-combine(1, 1, 2, 3, label: $+$,),
)
```



Paramètres

```
col-combine(
  row: int,
  col1: int,
  col2: int,
  to: int,
  label: content str,
  pos: auto str,
  bend: auto angle,
  flip: boolean,
  ..args: arguments dictionary
) -> content
```

row `int`

indice de la ligne (ou colonne si flip:true) de départ

col1 `int`

indice de la 1ère colonne de départ

col2 `int`

Indice de la 2ème colonne de départ

to `int`

uindice de la colonne d'arrivée

label `content` or `str`

texte/opération (ex: +12 ou + col 2)

Par défaut: `$+$`

pos `auto` or `str`

position : « top » (au-dessus) ou « bottom » (en dessous), défaut : auto

Par défaut: `auto`

bend `auto` or `angle`

courbure de l'arc

Par défaut: `auto`

flip `boolean`

pour l'utilisation dans les tableaux verticaux

Par défaut: `false`

..args `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

rdarrow
Flèche à droite allant vers le bas avec un label

Paramètres

```
rdarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne descendues

Par défaut: `1`

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

ruarrow
Flèche à droite allant vers le haut avec un label

Paramètres

```
ruarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne montées

Par défaut: `1`

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

ldarrow

Flèche à gauche allant vers le bas avec un label

Paramètres

```
ldarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne descendues

Par défaut: `1`

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

luarrow

Flèche à gauche allant vers le haut avec un label

Paramètres

```
luarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne montées

Par défaut: 1

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

urarrow

Flèche en haut allant vers la droite avec un label

Paramètres

```
urarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne descendues

Par défaut: 1

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

ularrow
Flèche en haut allant vers la gauche avec un label

Paramètres

```
ularrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne descendues

Par défaut: 1

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

drarrow
Flèche en bas allant vers la droite avec un label

Paramètres

```
drarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne descendues

Par défaut: 1

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

dlarrow

Flèche en bas allant vers la gauche avec un label

Paramètres

```
dlarrow(
  label: math.content,
  n: int,
  ..a: arguments dictionary
) -> content
```

label `math.content`

étiquette de la flèche

n `int`

le nombre de ligne descendues

Par défaut: `1`

..a `arguments` or `dictionary`

arguments transmis à la fonction edge de Fletcher

Sous-module pour les listes de couleurs et « hachures »

- nelines()
- grille()
- nwlines()
- alveoles()
- etoile()
- stripes()
- dots()
- cross()

Variables

- hachures
- typst18
- cycles-couleurs

nelines

North-east lines tiling

Paramètres

```
nelines(
  c: color,
  size: length,
  fill: color
) -> tiling
```

grille
grid lines tiling

Paramètres

```
grille(
  c: color,
  size: length,
  fill: color
) -> tiling
```

nwlines
North-west lines tiling

Paramètres

```
nwlines(
  c: color,
  size: length,
  fill: color
) -> tiling
```

alveoles
hexagons tiling

Paramètres

```
alveoles(
  c: color,
  size: length,
  background: color,
  fill: none color
) -> tiling
```

etoile
stars tiling

Paramètres

```
etoile(
  c: color,
  size: length,
  background: color,
  fill: none color
) -> tiling
```

stripes
colored stripes tiling that print in b&w

Paramètres

```
stripes(
  c: color,
  size: length,
  fill: auto color
) -> tiling
```

dots
colored dots tiling that print in b&w

Paramètres

```
dots(
  c: color,
  size: length,
  fill: auto color
) -> tiling
```

cross
colored cross tiling that print in b&w

Paramètres

```
cross(
  c: color,
  size: length,
  fill: auto color
) -> tiling
```

hachures `array`

List of tilings

typst18 `array`

colormap of basic typst colors

cycles-couleurs `array`

25 cycles de couleurs disponibles